

ATLETA REST API SPECIFICATION

Module: Authentication, Identity, Profiles & Social OAuth

• Authentication Standard: RFC 6750 Bearer Tokens (JWT). Authorization: Bearer <JWT_TOKEN>

Supports Athlete, Coach, and Official account creation

POST /users/register-athlete (Aliases: /users/register, /auth/register)

Public

Registers a new Athlete account in Firebase Auth and provisions both a User base entity and an Athlete_Profiles child document in Firestore.

Content-Type: [application/json](#)

Request Parameters / Body:

- first_name (string, required) - Athlete given name (max 255 chars)
- last_name (string, required) - Athlete family name (max 255 chars)
- email (string, required) - Valid unique email address (RFC 5322 compliant)
- password (string, required) - Account password (min 6 characters, recommend uppercase + symbol)
- role (string, required) - "Athlete" or "athlete"
- birthdate (string, required) - Birthdate formatted as YYYY-MM-DD
- gender (string, required) - "Male" or "Female"
- province (string, required) - Athlete province (e.g. "Camarines Sur")
- sport_type (string, required) - "Basketball", "Swimming", "Track and Field"
- contact_number (string, optional) - 11-digit mobile number (e.g. "09123456789")
- terms_accepted (boolean, required) - Must be true

Sample Response:

```
{
  "message": "User registered successfully.",
  "user": { "user_id": "s7PYR0Rcg0c8C...", "full_name": "Juan Dela Cruz", "role": "Athlete" },
  "profile": { "athlete_id": "ath_s7PYR0R...", "sport_type": "Basketball", "recruitment_status": "Available" },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Note: HTTP 201 on success. Returns full User object, Athlete profile, permissions array, and signed JWT token.

POST /users/register-coach (Aliases: /users/coach, /coaches/register)

Public

Registers a new Coach account with credential verification documents. Supports multipart/form-data for document upload.

Content-Type: [multipart/form-data](#) or [application/json](#)

Request Parameters / Body:

- first_name, last_name, email, password, contact_number (same as Athlete)
- role (string, required) - "Coach" or "coach"
- current_institution (string, required) - School or club name
- years_of_experience (number, required) - Integer from 0 to 60
- certification_license_num (string, optional) - Coach license ID
- eligible_documents / professional_documents (file, optional) - PDF or image up to 25MB

Sample Response:

```
{
  "message": "User registered successfully.",
  "user": { "user_id": "coach_abc123", "role": "Coach", "account_status": "Pending" },
  "token": "eyJhbGciOiJIUzI1Ni..."
}
```

Note: Account status is set to Pending until certified by System Admin.

POST **/users/login** (Aliases: **/auth/login**, **/api/auth/login**)

Ø<B Public

Authenticates any user role (Athlete, Coach, Official, Admin). Verifies credentials with Firebase Auth and retrieves role permissions from Firestore.

Content-Type: [application/json](#)

Request Parameters / Body:

- email (string, required) - Registered email address
- password (string, required) - User password

Sample Response:

```
{
  "message": "Login successful.",
  "user": {
    "user_id": "s7PYR0Rcg0c8C0WdkfnsqKMU3eK2",
    "first_name": "Juan",
    "last_name": "Dela Cruz",
    "email": "juan@example.com",
    "role": "Athlete"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "firebase_id_token": "eyJhbGciOiJSUzI1NiIs..."
}
```

Note: HTTP 200 on success. HTTP 401 on invalid email/password. Returns both Athlete JWT token and Firebase ID token.

POST **/users/social-login** (Aliases: **/users/google**, **/auth/google**, **/auth/facebook**)

Ø<B Public

Authenticates users using Google or Facebook OAuth ID tokens. If the user is logging in for the first time, automatically provisions their User and Athlete profiles.

Content-Type: [application/json](#)

Request Parameters / Body:

- id_token / token (string, required) - Firebase ID token or Google Access token from OAuth popup
- provider (string, optional) - "google" (default) or "facebook"
- role (string, optional) - "Athlete" (default) or "Coach"

Sample Response:

```
{
  "message": "GOOGLE login successful.",
  "user": { "user_id": "google_10283...", "full_name": "Dave Ildefonso", "role": "Athlete" },
  "token": "eyJhbGciOiJIUzI1Ni..."
}
```

Note: Handles both Login and Auto-Registration seamlessly. Cryptographically verifies token signature with Google/Firebase.

GET **/users/me** (Aliases: **/users/profile**, **/athlete/profile**, **/coach/profile**)

Ø=Ÿ Bearer JWT

Fetches the full profile, subtype profile metrics, and permissions of the user associated with the provided JWT bearer token.

Content-Type: [application/json](#)

Request Parameters / Body:

- Header: Authorization: Bearer <JWT_TOKEN>

Sample Response:

```
{
  "user": { "user_id": "s7PYR0...", "first_name": "Juan", "role": "Athlete" },
  "permissions": ["read:profile", "update:profile", "read:stats", "view:notifications"]
}
```

Note: HTTP 200 on success. HTTP 401 Unauthorized if token is missing, expired, or tampered.

POST**/users/password-reset (Aliases: /users/forgot-password, /auth/reset-password)**

Ø<ß Public

Generates a secure password reset token and dispatches an automated HTML email via Nodemailer containing the frontend reset URL.

Content-Type: [application/json](#)

Request Parameters / Body:

- email (string, required) - Registered email address

Sample Response:

```
{
  "message": "If an account with that email exists, a password reset link has been sent."
}
```

Note: Always returns HTTP 200 to prevent email enumeration attacks.

POST**/users/password-reset/confirm (Aliases: /users/reset-password, /auth/change-password)**

Ø<ß Public

Resets the user password using a verified token or authenticated session.

Content-Type: [application/json](#)

Request Parameters / Body:

- token (string, required if unauthenticated) - Password reset token from email link
- password (string, required) - New password

Sample Response:

```
{
  "message": "Password reset successfully. You may now log in with your new password."
}
```


